

1. UTF-32 & Character Encoding

- **UTF-32 Definition:** Encodes every character on **4 bytes** (32 bits) "as it is".
- **Pros:** Faster text operations (like character count) because every character is the same length.
- **Cons:**
 - **Super wasteful** in space (mostly zeros for English text).
 - Not compatible with ASCII.
 - Issues with "Endianness" (byte order).
- **Exam Tip:** When asked to represent a character in UTF-32, you must write all 4 bytes.
 - *Example:* 'A' (ASCII 65 or 0x41) in UTF-32:
 - 00000000 00000000 00000000 01000001.

2. Endianness (Byte Ordering)

Describes how a multi-byte value is stored in memory addresses.

- **Big Endian:** Stores the **MSB** (Most Significant Byte) first (in the lowest address).
 - *Used by:* Motorola 68k, PowerPC.
- **Little Endian:** Stores the **LSB** (Least Significant Byte) first (in the lowest address).
 - *Used by:* Intel x86, NIOS,.
- **Important Calculation (Dr. Shadenda Style):**
 - If you have 0x1122334455667788 stored at address 9656 in a **Little Endian** machine:
 - Byte 88 (LSB) 9656
 - Byte 77 9657
 - Byte 66 9658...
 - Byte 22 **9662**.

3. Number Systems & Conversions

- **Binary to Decimal:** Sum of weights ().
- **Decimal to Binary/Hex:** Repeated division (recording remainders),.

- **Hex Binary:** Group bits into **4s** (Nibbles).
 - 1A5F (Hex) = 0001 1010 0101 1111 (Binary),.
-

4. Signed Integers Representation

Three ways to represent negative numbers:

1. Sign-Magnitude:

- MSB is the sign (0 = Positive, 1 = Negative).
- *Problem:* Two zeros (+0 and -0),.

2. 1's Complement:

- Invert all bits.
- *Problem:* Still has two zeros.

3. 2's Complement (Preferred):

- Invert all bits + add 1.
- Only one zero representation.
- To represent -5: Take 5 (00000101), invert (11111010), add 1 11111011.

Sign Extension: When moving a signed number to a larger register (e.g., 8-bit to 16-bit), fill the new upper bits with the **Sign Bit** (MSB).

- Positive (0101) becomes 00000101.
 - Negative (1101) becomes 11111101,.
-

5. Fixed-Point Numbers

- Represented by $\langle w, b \rangle$ where **w** is total bits and **b** is the binary point position.
 - **Conversions:**
 - 0.45 Decimal to Binary Multiply by 2 and take the integer part (0.0111...).
 - **Negative Fractions:** Can be represented using 2's complement logic.
 - Example: -2.375.
 - Find +2.375 (0010.0110), flip bits (1101.1001), add 1 to LSB 1101.1010.
-

6. Arithmetic & Condition Codes (NVZC)

When performing arithmetic (ALU operations), the **Condition Code Register (CCR)** flags are updated:

- **N (Negative)**: Set to **1** if the result's MSB is 1.
- **Z (Zero)**: Set to **1** if the result is all Zeros.
- **V (Overflow)**: Set to **1** if the result is invalid (too big/small to fit).
 - *Rule*: Occurs if Adding (Pos + Pos = Neg) or (Neg + Neg = Pos).
 - *Note*: Overflow **never** occurs when adding a Positive to a Negative.
- **C (Carry)**: Set to **1** if there is an end carry out (or borrow in subtraction).

Exam Style Question: Calculate $0110 + 1100$ (4-bit 2's comp) and find flags.

- Result: 0010 (with carry out 1).
- **N=0** (MSB is 0), **Z=0** (Result not zero), **V=0** (Pos + Neg, no overflow), **C=1** (End carry).
- Format: x. NZVC 0010. 0001.